Intelligent Academic Path Optimizer

James Simpson[1], Anndeen Daley[2], Charles Varga[3], Tara Edney[4], Younes Roumilla[5], Bella Gerken[6]

Seidenberg School of Computer Science and Information Technology, Pace University
15 Beekman St, New York, NY 10038

[1]js81772p@pace.edu
[2]ad20140n@pace.edu
[3]cv64197n@pace.edu
[4]te34597n@pace.edu
[5]yr85580n@pace.edu
[6]bg29140n@pace.edu

***Abstract*— Students and Academic Advisors face the challenge of creating schedules that are best suited to student experience and learning style. Oftentimes, scheduling decisions are made on an ad hoc basis. The Intelligent Academic Path Optimizer seeks to solve that. IAPO is a web-based application that can assist students, advisors, and administrators in creating course schedules. As inputs, the program takes some parameters and potentially an undergraduate or graduate transcript. These parameters include options such as desired year to graduate and how many credit hours per semester to take. IAPO takes these inputs and uses AI to analyze them. The application then outputs a course schedule based on the parameters and uploaded transcript. This output is an optimized schedule for the student.**

***Keywords*— Academic Advising, Education, Information Technology.**

I. Introduction

A. *Why Build IAPO?*

Course selection for a postsecondary education is often approached by students and advisors without a great deal of formal analysis. While Academic Counseling departments typically provide holistic analyses for students when advising them on what courses to take, the advent of AI offers us the opportunity to utilize this growing technology to reduce workload on students, advisors and department heads.

B. *IAPO Components*

IAPO consists of a React website, a backend Python server, an AWS database and a connection to Google's Gemini AI. The website serves a user interface that allows the user to select a number of options, login to their account, and give IAPO specific custom instructions, such as "I don't want to take back-to-back courses in advanced math."

The backend Python server hosts the application's business logic, user account login, other internal functions, and acts as the middle man between the user and the database. Finally, the database holds information on newly created users as well as the course information the AI builds off of.

C. *Intended User Base*

The intended user base of IAPO are students, academic advisors, and administrators. This list is not exclusive, but inclusive and there may be other potential users that could find value from IAPO. However, these were the three users in mind when the product was conceived.

Students. Students may be confused about prereqs or graduation requirements. IAPO can provide the fastest path to graduate, "what-if" planning and an optimized schedule tailored to the student's needs, goals and experience.

Academic Advisors. Academic advisors will find use for IAPO in checking prereqs and holds instead of having to do so manually. This will allow Academic Advisors more time to help students and reduce the risk of human error.

Department Administrators. Administrators can use IAPO to assist in spotting bottlenecks and supporting data-driven decisions to raise graduation rates. The ways in which the application can be used by administrators is wide: it can assist with tasks ranging from simulating potential schedules to creating templates for student graduation paths.

D. *Minimum Viable Product*

The Minimum Viable Product will provide the basic functionality of the app to take inputs (including a transcript), analyze them with the AI, and give the user useful information.

It will contain a user-friendly interface, a text input in which students can specify custom academic needs to the AI, and an account login system so information can be saved.

The output will be a class schedule that students should take for a particular major as well as custom feedback from the AI on their schedule.

II. Literature Review

Academic courses can be linked to an overwhelming amount of data: topic, time, semester, location, availability, and so on. It can be a wall of pertinent information when one is deciding a class to choose for the semester. Further, all that information must also mesh with other classes being taken. For example, it makes no sense to have a class that ends at 3:00pm on one campus and another that starts at 3:05 on a different campus. Parsing transcripts manually is an incredibly lengthy task due to the diversity of documentation; as noted by Torkian and Zhou, admissions staff often spend up to 70%

of their time on manual data entry, leading to severe decision delays [1]. Before the most recent generative AI boom, optical character recognition was possible for data extraction in some documents; however, these still required large amounts of post processing and human labor [3]. Recent advances have overcome these extraction limits using multi-agent architecture and were able to achieve a 96.7% accuracy rate [1]. However, extracting transcript data addresses only half the administrative challenge. IAPO aimed to succeed at this transcript parsing and also create a functional agenda with this information. Scheduling by itself can be quite complicated for the aforementioned reasons. Ding, et al., note that directed heuristic-based methods and autonomous learning-based methods have been previously used for dynamic scheduling [2]. These processes can come with their own difficulties such as needing complete and perfect information which can be difficult to attain in a real environment. IAPO used Google Gemini as well as CSP solving algorithms to bridge the gap between these issues. By pairing AI parsing with dynamic constraint satisfaction, IAPO converts static academic history into optimal schedules.

III. Systems Architecture

The IAPO is made of four primary components: a web frontend, utilizing React, a code backend, hosted on a PythonAnywhere server, a storage database, hosted on the AWS cloud, and the Google Gemini API. Fig. 1 represents the sequence of events that occurs starting from user initialization to the schedule generated and displayed for them. The data that the backend receives from the user interacting with the web frontend ultimately determines whether the data gets sent to the Gemini API or to the AWS database. If the user is sending login or registration information to the backend, then the data gets forwarded to the database to check against existing data for login or account registration purposes. If the user is attempting to interact with the AI feature, then the data gets forwarded to the Gemini API to return a response. As for the different technologies that we decided to rely on for the application, we picked the Gemini API because it has a free tier which can be used up to a certain limit, which makes it best suited for a minimum viable product. We opted for the React framework as our frontend because it provided a flexible, component-based approach to building the user interface. For the backend, we chose FastAPI as the framework for its high execution speed and automatic validation of requests. We found it best to host our backend on PythonAnywhere due to its quick setup and free hosting. For deploying the website, we utilized GitHub Pages on a separate repository, as we had limited administrative access to the GitHub repository where the application is hosted. This allows it to be publicly accessible without having users download and build the application themselves. With regards to the AWS database, we used a DynamoDB for its NoSQL structure, easing the use of updating and viewing our data. We additionally used S3 buckets as a way to store large form user data, specifically uploaded transcripts. These databases were then policy protected through Amazon's Identity and Access Management (IAM) ensuring that only valid users can access the data.

Data Flow Diagram

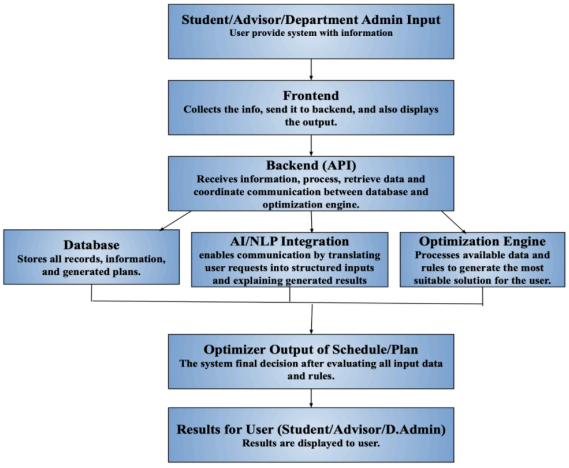

Fig. 1 A high-level view of the structure of IAPO.

IV. Frontend Architecture

The frontend for the IAPO was developed with JavaScript and React. Figma was used to create a mock-up of what we wanted the site to look like. We then used Vite, a local development server, which allowed us to view and test the site as it was being built. Vite converts the JSX code into plain JavaScript so that the browser can read and understand what to display.

React by itself is a single-page application framework that will only display one HTML page. We used the react-router-dom, a React library which allows for faster rendering through URL updates directly in the browser's address bar without a full page reload from the server. The home page is initially loaded and gives the option to either sign up or log in. After signing up, the user is able to put in commonly requested information about themselves as well as information specific to the app such as: major, credits for future semester, degree level, etc. This form is managed with useState, a built-in React hook, and allows the corresponding variable to update instantly. The `UserContext.jsx` file is used to retrieve profile information from the backend and save it to localStorage so we can reference the user's login status, application data, and profile information without having to make additional network calls.

We also used EmailJS which allowed us to receive emails from consumers using JavaScript. Through the Contact page, one could fill out the template to suggest changes or bug fixes and the message is sent directly to us as an email. This mechanism alongside speed checks, story points, and cross-browser tests, were used to ensure the user had an ideal experience. Fig. 2 displays one of the performance checks that were analyzed for quality assurance.

Lighthouse Quality Test

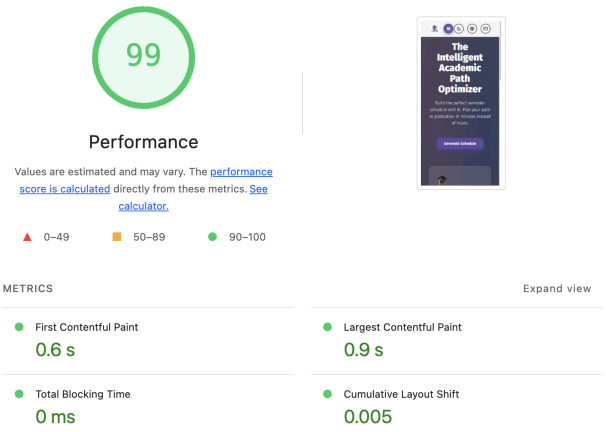

Fig. 2 Performance and speed check

V. Backend

The backend for the IAPO utilizes the Python language and FastAPI framework, hosted on PythonAnywhere. Due to PythonAnywhere not natively supporting FastAPI, a custom virtual environment was created and used to install all the necessary components to ensure that FastAPI worked. The backend itself serves as the middleman between the user, AWS database, and Gemini API. This provides an additional layer of security by preventing the frontend from directly accessing sensitive information such as user credentials or API keys. Additionally, as an extra layer of security, the backend can only be accessed when a valid API key is included with each request. This separation also makes the individual components of the application more independent of one another. For example, the frontend can be modified or deployed to a different location without requiring changes to the backend. Similarly, if the database is updated or changed, the backend can handle these changes without requiring modifications to the frontend, allowing the application to remain flexible and easier to maintain.

The backend follows a top-down structure, similar to the organization of the frontend. Rather than placing all functionality within a single file, the backend is divided into separate files based on their respective responsibilities. This modular structure allows individual components to be updated without unnecessarily affecting higher-level functionality. The main script initializes the FastAPI framework and establishes the authentication requirements for incoming requests. From there, requests are directed to the routing script, which defines the individual endpoints used by the frontend. Each endpoint then delegates the request to the appropriate module based on its functionality. For example, requests involving user information, such as creating or updating a user, are handled by a dedicated user module containing the relevant functions. Similarly, requests involving AI generation are directed to a separate module responsible for communicating with the Gemini API. Additional helper files were also created to support the backend by defining request schemas, configuring the connection to the AWS database, and handling credential validation. These include functions for validating passwords and verifying the API key included with incoming requests. Separating these responsibilities keeps the backend organized and ensures that common functionality can be reused across multiple endpoints without duplicating code.

VI. AI

The AI Extraction Layer has been developed using Python utilizing a FastAPI web development framework to define API endpoints and communicate with the Frontend. Requests to the Language Model are issued through the Google Gemini API via the `google-genai` SDK. The `gemini-3.1-flash-lite` model was selected due to its ability to provide low latency performance on structured extraction tasks. Six primary Python modules exist with the extraction layer application. API credentials are loaded at runtime from environment variables using the `python-dotenv`, keeping keys out of version control.

Users are provided the functionality of uploading an academic transcript, which the Frontend transmits as a base64 encoded data url. The uploaded file is decoded and parsed server-side using the `pypdf` library, and the resulting extracted text is supplied to the model as additional context.

Pydantic is used to define all schemas that structure model output within the system. Rather than parsing free-form text, the `StudentPlanRequest` model is passed to the Gemini API as a Response Schema utilizing a JSON Response Type, thus constraining generation to a valid structure and eliminating malformed output. The same models validate all data returned to the Solver to ensure type and range correctness.

VII. Database & Implementation

A. Database Architecture & Choice

For the backend of the Intelligent Academic Path Optimizer (IAPO), Amazon DynamoDB was selected as the primary database and is hosted through Amazon Web Services (AWS). DynamoDB is a fully managed, serverless NoSQL key-value and document database service, which brings more flexibility and easy integration with backend and AI optimizer.

B. SQL Database Development

The database was originally developed using SQLite, managed through Dbeaver, a database management tool used to create, organize, and test the relational database. Although SQLite was not used as the finalized database host, it assisted with the development of the entities and relationships in the IAPO system. The database included tables for courses, majors, prerequisites, courses offered, major courses, semesters etc. Primary and foreign keys were used to maintain relationships between related entities. For example courses connecting to prerequisites or StudentID (primary key) being included when it comes to scheduling preferences located in Student Constraints. The SQL development provided an organized representation of the system's data and used queries to test the database design. This process was useful for

identifying entities, attributes and relationships required by the system before transitioning to a NoSQL implementation. Fig. 3 illustrates how these blocks of information are structured to relate to one another.

Database Design

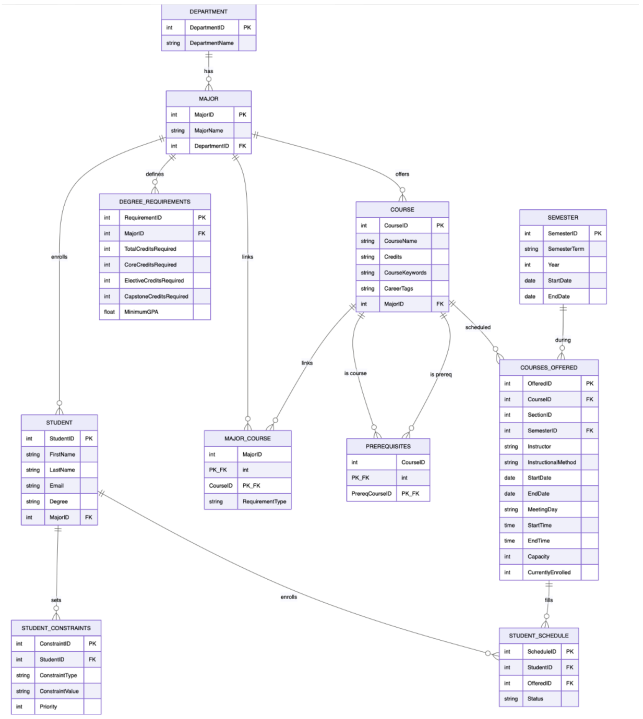

Fig. 3 Entity relationship diagram for database visualization

C. NoSQL Database Implementation

Amazon DynamoDB was used in the development of the NoSQL implementation. The DynamoDB data model was developed based on the functional requirements and expected data needs of the IAPO system, ensuring that the database structure could support the information required by the backend and scheduling components.The database was designed around the expected ways the system will retrieve and use data, allowing the table and key structures to support the application's required data retrieval operations. Unlike the SQL implementation, which relied on primary and foreign keys to establish relationships between tables, the DynamoDB implementation was designed around flexible item structures using partition keys and sort keys that allowed related items to be identified and retrieved efficiently without relying on relational connection. DynamoDB items were recorded in JSON format, a lightweight text format used to store and send data. It can be exported into csv files to become easier to read and analyze amongst team members. The database included data for courses, course offerings, students, majors, prerequisites, semesters etc. The NoSQL structure also provided flexibility when it comes to student scheduling constraints, allowing the database to adjust to different types of scheduling preferences without requiring significant change to the database structure.

D. Backend & AI Integration

DynamoDB also supports the interaction between the backend and AI Optimizer. Students can provide requests through plain text, which the AI component can interpret the information needed to fulfill the request. Depending on the request, the information can include data from either of the tables located within the database and uploaded transcripts. The backend can also use the interpreted request to retrieve and provide the relevant data from DynamoDB to the appropriate system components. The results are returned through the backend and stored in DynamoDB for continued access and use by the system.

E. Database Summary

Overall, the SQL and NoSQL implementations provided a foundation for developing and evaluating the database requirements of IAPO. With DynamoDB operating as the data layer, the backend manages communication between components while the AI and optimization components process requests and generate results. Fig. 4 depicts an example of what is produced and displayed for the user.

Generated Schedule

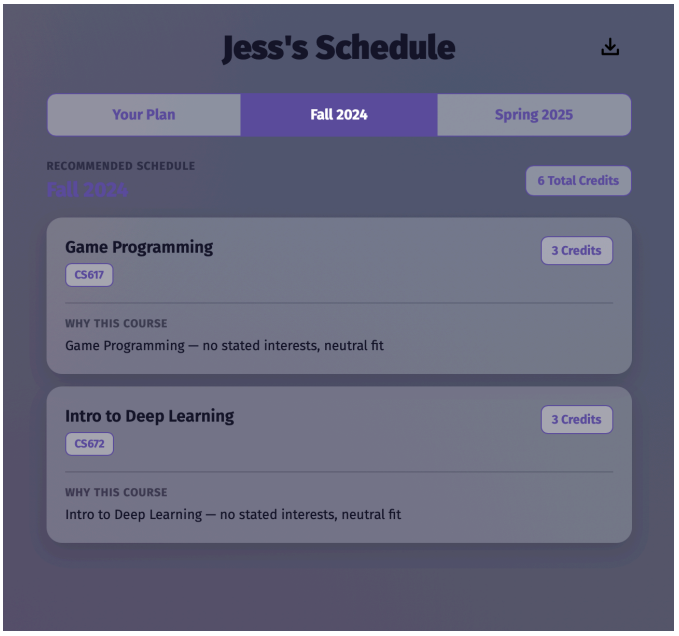

Fig. 4 Example of a schedule created using IAPO

VII. Summary & Conclusions

A. Results

The IAPO program worked as intended. Users can set up an account, log in, and request a schedule. The system will return a schedule as intended. Further, all forms are properly validating input. For example, when a user is creating an account, if they don't enter a valid email, the site will give them an error message.

B. Implications and Recommendations

IAPO shows that AI can be used to assist in student scheduling, beyond a simple interaction with a chatbot. The implication is this: perhaps the future of AI lies not only in developing more powerful models, but in developing systems to do interesting things with AI.

This is similar to how NetApp is not really a hardware company even though they sell storage devices. Their key features have usually been software: How block level backups, RAID and other features like deduplication work is in many ways more important than the hardware they are run on.

C. Conclusions

In summary, IAPO delivers a student's optimized schedule by checking both course information and using AI to analyze a student's transcript to further tailor a schedule to their needs. IAPO is part of a potential trend that could lead to innovation in AI originating not in making better language models, but in creating the tools that utilize those LLMs in a unique way to do something useful.

References

[1] B. Torkian and J. Zhou, "A multi-agent AI system for automated high school transcript processing: Collaborative document analysis at scale," arXiv preprint arXiv:2606.13916, Jun. 2026.

[2] J. Ding et al., "A survey of AI-Enabled Dynamic Manufacturing Scheduling: From directed heuristics to Autonomous Learning," *ACM Computing Surveys,* vol. 55, no. 14s, pp. 1–36, Jul. 2023.

[3] T. Hegghammer, "OCR with tesseract, Amazon Textract, and Google Document AI: A benchmarking experiment," *Journal of Computational Social Science,* vol. 5, no. 1, pp. 861–882, Nov. 2021.

[4] Redgate Data Modeler, "5 Important Database Concepts for the Software Engineer," Redgate, Feb.22.2022. https://www.red-gate.com/blog/database-concepts-for-software-engineer/.

[5] GeeksforGeeks, "SQL vs. NoSQL," GeeksforGeeks, Jun.22.2026. https://www.geeksforgeeks.org/sql/difference-between-sql-and-nosql